

Advanced Front-End with React + Front-End/Back-End Communication

Lecture 4.

React Basics + SPA + API Communication

Plan

React fundamentals (components, props, state)

Events & user interaction

Lists + conditional rendering

React Router (SPA)

Fetch/Axios

JSON handling

Loading + errors

JWT authentication basics

Learning Outcomes



By the end of this lecture students can:



Build dynamic React interfaces



Manage state and props correctly



Build a SPA using React Router



Communicate with REST APIs



Handle async data, loading and errors



Understand JWT authentication basics

React Core Concepts

What is React?

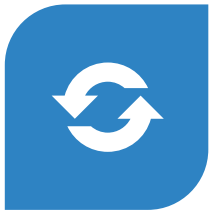
React is a JavaScript library for building user interfaces (UI)
It helps developers create fast, interactive applications.

React is component-based
You build the UI from small independent pieces.

React uses declarative rendering
You describe the UI for a given state — React updates the screen automatically.

React is ideal for Single Page Applications (SPAs)
Pages feel instant because React updates parts of the UI without full reloads.

Why React is Popular



**REUSABLE
COMPONENTS REDUCE
REPEATED CODE**



**STRONG ECOSYSTEM
(ROUTER, LIBRARIES,
TOOLS, UI KITS)**



**FAST RENDERING
USING VIRTUAL DOM
UPDATES**



**LARGE COMMUNITY →
TUTORIALS, SUPPORT,
JOBS**

React App Structure

Typical project structure:

- ▶ `src/` → main code
- ▶ `components/` → reusable UI blocks (Button, Card, Navbar)
- ▶ `pages/` → full pages (Home, Login, Profile)
- ▶ `App.jsx` → main app layout and routing
- ▶ `main.jsx` → React entry point (mounts the app to HTML)

Component Concept

A component is:

- ▶ **A reusable piece of UI**
- ▶ Usually written as a **function that returns JSX**

Can contain:

- ▶ UI (JSX)
- ▶ logic (JavaScript)
- ▶ state (useState)
- ▶ side effects (useEffect)

Functional Components

Modern React uses **functional components**

- ▶ A functional component is just a function:
 - ▶ receives props
 - ▶ returns JSX
- ▶ Example idea:
 - ▶ `function Header() { return <h1>Hello</h1> }`

JSX Basics

JSX is:

- ▶ A syntax extension that looks like HTML inside JavaScript
- ▶ Makes UI code readable and structured
- ▶ Allows mixing UI + logic:
 - ▶ `{2 + 2}`
 - ▶ `{user.name}`
 - ▶ `{isLoggedIn ? ... : ...}`

JSX Rules

- ▶ Important JSX rules:
- ▶ Must return **one parent element**
(wrap with `<div>` or `<>...</>`)
- ▶ Use `className` instead of `class`
- ▶ Use camelCase attributes:
 - ▶ `onClick`, `onChange`, `tabIndex`
- ▶ Self-close tags:
 - ▶ `<img />`, `<input />`

Props: Concept

Props (properties) are:

- ▶ **Data passed from parent → child**
- ▶ Used to make components reusable
- ▶ Props are **read-only**
(a child should not change parent data)

Props Example

Parent passes values:

- `<UserCard name="Anna" age={22} />`

Child receives them:

- `function UserCard(props) { ... }`

Props allow:

- one component design
- multiple variations of content

Props Best Practices

Keep props:

- clear
- descriptive
- minimal

Avoid “prop drilling” (too many levels)

If many props are needed:

- consider splitting components
- consider Context later

State: Concept

State is:

- ▶ **Data stored inside a component**
- ▶ Used when data can change:
 - ▶ user input
 - ▶ UI toggles
 - ▶ API results
- ▶ Updating state triggers a **re-render**

useState Hook

Hook syntax:

- `const [count, setCount] = useState(0)`
- `count` → current value
- `setCount()` → updates state and re-renders UI

State updates should be done using setter,
not direct assignment

React re-renders a component when:

- ▶ State changes
- ▶ Props change
- ▶ Parent re-renders and passes new props
- ▶ React does NOT reload the page — it updates only UI parts.



Re-rendering

State vs Props

Props	State
Passed from parent	Inside component
Read-only	Can be updated
Makes component reusable	Makes component dynamic

Props:

external input
passed from parent
read-only

State:

internal component data
controlled by component
can change

Both control what is displayed.

State is used for:

- ▶ form inputs (email, password)
- ▶ UI toggles (isOpen, darkMode)
- ▶ lists (products, users)
- ▶ async status (loading, error)
- ▶ authentication (token, user)

Common State Examples

React handles events similar to HTML but in JSX:

- ▶ `onClick`
- ▶ `onChange`
- ▶ `onSubmit`
- ▶ `onKeyDown`
- ▶ `onMouseEnter`

Event handlers are usually functions inside the component.

Events & Forms

Correct:

- `<button onClick={handleClick}>Save</button>`

Incorrect:

- `<button
onClick={handleClick()}>Save</button>`

- Because calling `handleClick()` executes immediately during rendering.

onClick Example

React passes an event object:

- ▶ `event.target.value`
- ▶ `event.preventDefault()`
- ▶ `event.stopPropagation()`

Used heavily in forms and input handling.

Event Object

Controlled input means:

- ▶ input value is stored in state
- ▶ React controls the value
- ▶ input updates via onChange

This is the standard approach in React.

Controlled Inputs

Controlled Input Example

Typical pattern:

```
▶const [email, setEmail] = useState("")
```

```
▶<input value={email}  
  onChange={(e)=>setEmail(e.target.value)}  
/>
```

This ensures React always knows the current value.

Form Submit

React form submit:

- ▶ `<form onSubmit={handleSubmit}>`
- ▶ `handleSubmit(e):`
 - ▶ `e.preventDefault()`
 - ▶ validate input
 - ▶ send API request

Why Controlled Forms Matter

A **controlled form** is a form where **React state** is the “single source of truth” for input values.

Why it matters:

- ▶ **Validation becomes easy** (you can validate on every keystroke)
- ▶ **Error messages become simple** (show errors based on state)
- ▶ **Form submission is predictable**
- ▶ **Input values can be reset easily**
- ▶ **Works perfectly with API calls** (send state as JSON)

Real-world example:

- ▶ login form
- ▶ registration
- ▶ checkout form
- ▶ profile edit page

Lists & Conditional Rendering

A **list rendering** means generating UI elements from an array of data.

- ▶ **Key concept:**
- ▶ React uses JavaScript `.map()` to transform data into JSX.

Why it's important:

- ▶ Most real apps display data:
 - ▶ products
 - ▶ posts
 - ▶ users
 - ▶ comments

Typical structure:

- ▶ `array` → `.map()` → component

Key Prop

A **key** is a special prop React uses to uniquely identify list items.

Why keys exist:

- ▶ React needs to know:
 - ▶ which item changed
 - ▶ which item was removed
 - ▶ which item was added
- ▶ Keys improve performance and prevent UI bugs.

Rule:

- ▶ Keys must be **unique and stable**.

Bad vs Good Keys

Bad:

- ▶ `key={index}`

Good:

- ▶ `key={user.id}`

Conditional Rendering

Conditional rendering means displaying different UI depending on a condition.

Common ways in React:

- ▶ if statement
- ▶ condition && `<Component />`
- ▶ ternary operator:
 - ▶ `condition ? <A /> : <B />`

Used for:

- ▶ loading
- ▶ errors
- ▶ authentication
- ▶ empty states

Typical UI logic:

If data is loading →
show spinner

If error → show
error message

If success → show
content

Why it matters:

Without conditions,
UI will:

- flash empty screens
- show broken layouts
- confuse the user

Common UI Conditions

Most real apps use conditions for:

- ▶ **Auth state**
 - ▶ logged in vs guest
- ▶ **Permissions**
 - ▶ admin vs user
- ▶ **Data availability**
 - ▶ empty list vs results
- ▶ **API status**
 - ▶ loading vs loaded vs error
- ▶ **UI state**
 - ▶ modal open/close

React Router (SPA)

A **Single Page Application (SPA)** is a web app that loads one HTML page once and updates the UI dynamically.

Key idea:

- ▶ Navigation happens without full reload.

Benefits:

- ▶ Faster transitions
- ▶ Smoother user experience
- ▶ Feels like a mobile app

Examples:

- ▶ Instagram web
- ▶ Gmail
- ▶ Netflix web

Why Use React Router

React itself does not include routing.

React Router allows:

- ▶ Multiple pages inside a SPA
- ▶ URL navigation:
 - ▶ /home
 - ▶ /products
 - ▶ /login
- ▶ Browser history support
- ▶ Dynamic pages with parameters

Router Core Components

Main React Router components:

- ▶ `<BrowserRouter>`
Enables routing in the app
- ▶ `<Routes>`
Container for all routes
- ▶ `<Route>`
Connects URL path to a component

Concept:

- ▶ URL → component → page UI

Basic Routing Example

- ▶ Example routes:

- ▶ / → Home page

- ▶ /about → About page

- ▶ /products → Products list

- ▶ /login → Login page

- ▶ Why it matters:

- ▶ A SPA still needs navigation and structure like a normal website.

Link vs a

Important rule:

Use `<Link>` instead of `<a>` in React Router.

Why:

- ▶ `<a>` triggers full page reload
- ▶ `<Link>` updates URL without reload
- ▶ Keeps app fast and SPA behavior correct

Dynamic Routes

A dynamic route includes a variable segment, such as:

- ▶ `/users/:id`
- ▶ `/products/:productId`

Purpose:

- ▶ Display a page for a specific item.

Example:

- ▶ `/users/15` → user profile page

Navigation Programmatically

Programmatic navigation means redirecting the user using code.

Use:

`useNavigate()`

Common cases:

- ▶ after login → go to dashboard
- ▶ after logout → go to login
- ▶ after form submit → go to success page

404 Page

A 404 page is used when:

- ▶ the user enters an unknown URL
- ▶ the route does not exist

Implementation:

- ▶ Catch-all route: `path="*"`

Why it matters:

- ▶ Prevents blank screens
- ▶ Looks professional

Front-End & Back-End Communication

Modern React apps are usually **data-driven**.

- ▶ React needs APIs for:
- ▶ user accounts
- ▶ posts/comments
- ▶ product catalogs
- ▶ orders and payments
- ▶ dashboards and analytics

Without API communication:

- ▶ UI becomes static and useless.

API (application programming interface)

API (Application Programming Interface) is a set of rules that allows applications to communicate with each other.

- ▶ Defines available requests and operations
- ▶ Acts as a gateway for data and service interaction

Why APIs are used:

- ▶ **Automation:** reduce manual work and improve workflow efficiency
- ▶ **Easy setup & customization:** add, remove, or modify features as needed
- ▶ **Flexible data access:** faster and more efficient information delivery
- ▶ **Easy integration:** enables different systems and platforms to work together

Common API protocols:

- ▶ SOAP
- ▶ REST
- ▶ GraphQL

SOAP

SOAP (Simple Object Access Protocol) is a messaging protocol for web services.

- ▶ Used to exchange structured data
- ▶ Based on **XML** with strict schemas (strongly typed)
- ▶ Each operation has a clearly defined XML request and response
- ▶ Supports simple and complex data types
- ▶ Platform- and language-independent
- ▶ Works over standard protocols like **HTTP**
- ▶ Commonly used for authentication, authorization, and system integration

REST

REST (Representational State Transfer) is an architectural style for distributed systems.

Core REST constraints:

- ▶ **Client-Server:** separates UI and data storage, improving portability and scalability
- ▶ **Stateless:** each request contains all required information; session state is stored on the client
- ▶ **Cacheable:** responses must define whether they can be cached
- ▶ **Uniform Interface:** simplifies architecture through clear resource identification and self-descriptive messages
- ▶ **Layered System:** supports hierarchical layers; each component interacts only with adjacent layers
- ▶ **Code on Demand (optional):** clients can extend functionality by executing code from the server

GraphQL

GraphQL is a query language for APIs that lets clients request exactly the data they need.

- ▶ Provides a clear and complete data schema
- ▶ Returns predictable and precise results
- ▶ Faster and more stable data fetching
- ▶ Allows querying related resources in one request
- ▶ Reduces over-fetching and under-fetching
- ▶ Performs well even on slow networks
- ▶ Organized by **types and fields**, not endpoints

SOAP vs REST vs GraphQL

Feature	SOAP	REST	GraphQL
Type	Protocol	Architectural style	Query language
Data format	XML only	JSON (mostly), XML	JSON
Flexibility	Low	Medium	High
Performance	Slower	Fast	Very fast
Endpoints	Fixed	Multiple endpoints	Single endpoint
Data control	Server-defined	Server-defined	Client-defined
Complexity	High	Low	Medium
Best use case	Enterprise, legacy systems	Public & web APIs	Complex data relationships

Why We Use REST API

REST APIs are popular because they are simple, scalable, and widely supported.

- ▶ **Easy to understand and implement**
- ▶ **Stateless** → better performance and scalability
- ▶ **Uses HTTP methods** (GET, POST, PUT, DELETE)
- ▶ **Works with JSON**, which is lightweight and readable
- ▶ **Platform- and language-independent**
- ▶ **Great for web and mobile applications**
- ▶ **Strong caching support** improves speed
- ▶ **Large ecosystem & community support**

REST is ideal when:

- ▶ You need a **simple and reliable API**
- ▶ Data structure is not highly complex
- ▶ Performance and scalability matter
- ▶ You want easy integration across systems

Web service

A **web service** is software available over the Internet that enables communication between different systems.

- ▶ Allows devices and applications to communicate via the **WWW**
- ▶ Uses standard web technologies such as **HTTP**
- ▶ Exchanges machine-readable data in **XML** or **JSON**
- ▶ Provides an interface to backend systems or databases
- ▶ Used by web and mobile applications
- ▶ Enables data sharing (syndication) across platforms

Difference between an API and a Web service

The main difference between these two :

1. The API acts as an interface between different applications, allowing them to interact with each other in a more efficient way.
2. A web service is designed to facilitate communication between two devices or machines over a network.

However, **other differences between the API and the web service** are :

- ▶ All web services can be used as an API, while you cannot use every API as a web service.
- ▶ The web service cannot perform all the operations that you can do using the API.
- ▶ The API can use any communication style.
- ▶ The web service always requires a network connection for normal operation, but the API does not.
- ▶ APIs usually contain interfaces and classes, just like a program.
- ▶ The API can help programmers establish communication between different software applications.

- **API** = broad concept for software communication
- **Web Service** = API exposed over the web using standard web protocols

API	Web Service
A general interface that allows software components to communicate	A type of API that works over the Internet
Can be local or remote	Always network-based
May use any protocol or format	Uses web standards like HTTP
Can exchange data in various formats	Typically uses XML or JSON
Not all APIs are web-based	All web services are APIs

REST API

REST (Representational State Transfer) is an architectural style for building web services.

- ▶ Resources are represented in text formats like **JSON**, **XML**, or **HTML**
- ▶ Resources are accessed and modified using standard **HTTP methods** (GET, POST, PUT, DELETE)
- ▶ Common operations follow **CRUD**: Create, Read, Update, Delete
- ▶ REST APIs use **HTTP** to define endpoints, requests, and responses
- ▶ An **API** defines how software components communicate with each other

REST API Basics

A REST API is a system where the front-end communicates with back-end through URLs (endpoints) and HTTP methods.

Core methods:

- ▶ GET → read
- ▶ POST → create
- ▶ PUT/PATCH → update
- ▶ DELETE → remove

Key concept:

- ▶ Front-end sends request
- ▶ Back-end sends response

REST as CRUD

HTTP command	Database operation	/dogs	/dogs/3
GET	<u>R</u> ead	List all dogs	Get dog details
POST	<u>C</u> reate	Create new dog	—
PUT	<u>U</u> pdate	—	Update detail/s
DELETE	<u>D</u> elete	Delete all dogs	Delete this dog

Активация Windows

Чтобы активировать Windows, перейдите к па

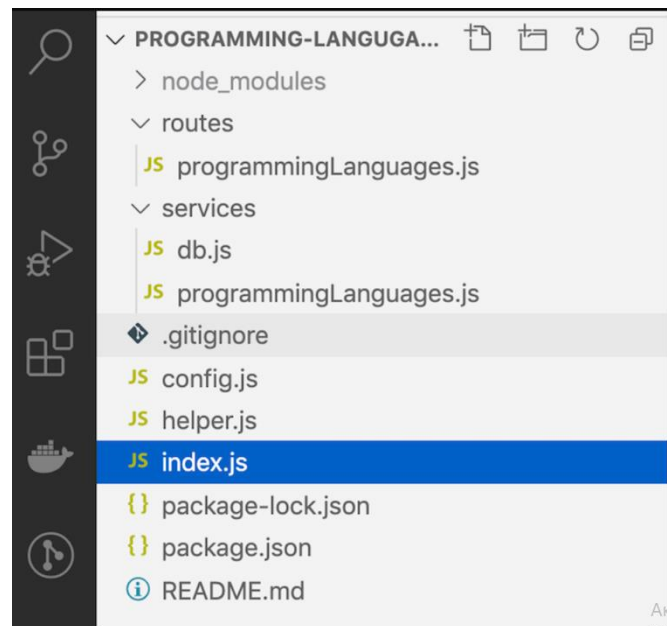
REST API project structure

`config.js` will contain configuration for information like the database credentials and the rows we want to show per page when we paginate results.

`helper.js` is the home for any helper functions, like calculating offset for pagination.

The `routes/programmingLanguages.js` file will act as the glue between the URI and the corresponding function in the `services/programmingLanguages.js` service.

The `services` folder will house all our services. One of them is `db.js`, which used to talk with the MySQL database.



REST GET Example 1

GET `http://dog-db.com/api/dogs`

```
[  
  { id:1, name:"Fido" },  
  { id:2, name:"Rover" },  
  { id:3, name:"Spot" },  
  { id:4, name:"Fluffy" },  
]
```

Активация Windows

REST GET Example 2

GET `http://dog-db.com/api/dogs/3`

```
{  
  id:3,  
  name:"Spot",  
  dob:"2009-05-21",  
  type:"spaniel",  
  photo:"http://dog-db/images/..."
```

Активация Windows

Чтобы активировать Windows, перейдите к п...

Expressing relationships

```
{  
  id:3,  
  name:"Spot",  
  dob:"2009-05-21",  
  owner:{  
    id:16,  
    name:"Sam",  
    url:"http://dog-db.com/api/owners/16"  
  }  
  ...  
}
```

Активация Windows

Чтобы активировать Windows, перейдите к [настройкам](#) компьютера.

HTTP response codes

HTTP code	Meaning
200	OK
4xx	Bad request (client's fault)
5xx	Failed request (server's fault)
401	Unauthorized request
404	Resource not found
500	Internal error (bug)
503	Server overloaded

JSON Format

JSON (JavaScript Object Notation) is the standard format for sending data between client and server.

Why JSON is used:

- ▶ lightweight
- ▶ readable
- ▶ easy to parse in JS

Typical response:

- ▶ objects { }
- ▶ arrays []

JSON

JSON stands for **JavaScript Object Notation**.

JSON is a **text-based data format** used for:

- ▶ storing data
- ▶ exchanging data between systems (e.g. frontend ↔ backend)

JSON is:

- ▶ **human-readable**
- ▶ **self-describing**
- ▶ **language-independent** (used not only in JavaScript)

```
let personAge = obj.age;
```

```
{  
  "name": "John",  
  "age": 30,  
  "car": null  
}
```

Accessing JSON Data in JavaScript

JSON Object Structure

The JSON object above contains **three properties**:

- `name` → `"John"`
- `age` → `30`
- `car` → `null`

Each property consists of:

- a **key** (string)
- a **value** (string, number, boolean, null, array, or object)

When a JSON string is **parsed** in JavaScript, it becomes a regular object.

```
let personName = obj.name;
```

This line accesses the value of the `name` property.

Why JSON is Important in Web Development

JSON is widely used in:

- ▶ REST APIs
- ▶ Frontend-backend communication
- ▶ Configuration files
- ▶ Data exchange between different technologies

In full-stack applications:

- ▶ The **backend sends data as JSON**
- ▶ The **frontend receives and processes JSON**

JSON and XML: Data Interchange Formats

JSON (JavaScript Object Notation)

JSON is a lightweight **data-interchange** format that is **completely language-independent**.

Although it is based on the JavaScript programming language, JSON can be generated and parsed by almost any modern programming language.

Key characteristics of JSON:

- ▶ Text-based and human-readable
- ▶ Easy to understand and generate
- ▶ Used mainly for **data exchange**, not for document formatting
- ▶ Defines **only a data encoding specification**
- ▶ Widely used in REST APIs and web applications
- ▶ JSON is especially popular in modern web development because it is:
 - ▶ Compact
 - ▶ Fast to parse
 - ▶ Naturally compatible with JavaScript and frontend frameworks such as React

XML (Extensible Markup Language)

XML was originally designed to **store and transport data**, not to display it.

XML is a **markup language** that defines a set of rules for encoding documents in a format that is both:

- ▶ Human-readable
- ▶ Machine-readable

Key characteristics of XML:

- ▶ Textual data format
- ▶ Strong support for Unicode (multiple human languages)
- ▶ Highly structured and self-descriptive
- ▶ Designed with simplicity, generality, and Internet usability in mind
- ▶ Although XML focuses on document representation, it is widely used for:
- ▶ Web services
- ▶ Configuration files
- ▶ Representation of complex and hierarchical data structures

JSON	XML
It is JavaScript Object Notation	It is Extensible markup language
It is based on JavaScript language.	It is derived from SGML.
It is a way of representing objects.	It is a markup language and uses tag structure to represent data items.
It does not provides any support for namespaces.	It supports namespaces.
It supports array.	It doesn't supports array.
Its files are very easy to read as compared to XML.	Its documents are comparatively difficult to read and interpret.
It doesn't use end tag.	It has start and end tags.
It is less secured.	It is more secured than JSON.
It doesn't supports comments.	It supports comments.
It supports only UTF-8 encoding.	It supports various encoding.

Employees Data Structure

JSON Example

```
{
  "employees": [
    { "name": "John", "age": 30, "department": "IT" },
    { "name": "Anna", "age": 25, "department": "HR" },
    { "name": "Peter", "age": 40, "department": "Finance" }
  ]
}
```

Both **JSON** and **XML** can be used to receive data from a web server.

However, in modern web development:

- **JSON** is the dominant format for REST APIs
- **XML** is more common in legacy systems and document-based services

XML Example

```
<employees>
  <employee>
    <name>John</name>
    <age>30</age>
    <department>IT</department>
  </employee>
  <employee>
    <name>Anna</name>
    <age>25</age>
    <department>HR</department>
  </employee>
  <employee>
    <name>Peter</name>
    <age>40</age>
    <department>Finance</department>
  </employee>
</employees>
```

Fetch API Overview

Fetch API is the built-in browser method for making HTTP requests.

Key points:

- ▶ returns a Promise
- ▶ you must parse JSON manually:
 - ▶ `response.json()`

Important detail:

- ▶ fetch does NOT automatically throw error on HTTP 404/500
→ you must check `response.ok`

Axios Overview

Axios is a popular HTTP library for JavaScript.

Advantages:

- ▶ auto-parses JSON
- ▶ better error handling
- ▶ supports interceptors
- ▶ cleaner syntax

Used often in professional React apps.

Fetch vs Axios

Fetch:

- ▶ built-in
- ▶ more boilerplate
- ▶ must handle errors manually

Axios:

- ▶ external library
- ▶ simpler code
- ▶ strong error handling
- ▶ better for JWT (interceptors)

Async Data, Loading, Errors

Asynchronous means code runs without blocking the rest of the program.

API requests are async because:

- ▶ network delay
- ▶ server processing
- ▶ database queries

Result:

- ▶ UI must wait and react to different states.

useEffect for API Calls

useEffect() is a hook for running side effects:

- ▶ API calls
- ▶ subscriptions
- ▶ Timers

Typical usage:

- ▶ fetch data when the component mounts

Important:

- ▶ dependency array controls when effect runs.

Loading State

A loading state is a variable that tracks whether data is being fetched.

Why it matters:

- ▶ improves UX
- ▶ prevents blank screen
- ▶ prevents user confusion

Example UI:

- ▶ spinner
- ▶ skeleton loader
- ▶ “Loading...” text

Error State

An error state stores information about a failed request.

Why errors happen:

- ▶ server down
- ▶ wrong URL
- ▶ network issue
- ▶ invalid token

UI should show:

- ▶ clear message
- ▶ retry button (optional)

UI Pattern

Best practice: handle 3 states

- ▶ Loading
- ▶ Error
- ▶ Success

This makes UI predictable and professional.

Example UI Flow

Example:

- ▶ loading = true → show spinner
- ▶ success → show data
- ▶ error → show “Something went wrong”

Key concept:

User should always understand what is happening.

Avoid Infinite Loops

Common mistake:

- ▶ including fetched state in dependencies

Why it causes loop:

- ▶ `useEffect` fetches → sets state → state change triggers `useEffect` again

Solution:

- ▶ fetch once with `[]`
- ▶ or fetch when specific values change (search query, filter, id)

API

Authentication (JWT Basics)

Authentication is needed to:

- ▶ protect private data
- ▶ verify user identity
- ▶ control permissions

Examples:

- ▶ only logged-in users can post
- ▶ only admin can delete
- ▶ only owner can edit profile

What is JWT

JWT (JSON Web Token) is a token used to prove that the user is authenticated.

JWT usually contains:

- ▶ user ID
- ▶ role (optional)
- ▶ expiration time
- ▶ signature

Important:

JWT is **signed**, not encrypted.

JWT Flow

Step-by-step:

- ▶ User sends login request (email + password)
- ▶ Server validates credentials
- ▶ Server returns JWT
- ▶ Front-end stores JWT
- ▶ Front-end sends JWT in headers for protected requests
- ▶ Server validates token and returns data

Authorization Header

Standard:

Authorization: Bearer <token>

Why “Bearer”: It means:
“whoever has this token is allowed”

Where to Store JWT

Main options:

- ▶ **localStorage**
 - ▶ easy but XSS risk
- ▶ **sessionStorage**
 - ▶ safer, clears after closing tab
- ▶ **httpOnly cookie**
 - ▶ best security, JS cannot access it

Security Warning and Protected Routes

Never store sensitive data in JWT.
JWT can be decoded easily (base64).

Key security notes:

- ▶ JWT can be decoded → never store sensitive data inside
- ▶ Always use HTTPS
- ▶ Use short expiration times
- ▶ Use refresh tokens in real apps

Protected Routes

A protected route is a route that requires authentication.

- ▶ Logic:
- ▶ If token exists → show page
- ▶ If no token → redirect to login

Used for:

- ▶ dashboard
- ▶ profile
- ▶ admin panel

Summary

- how React components work
- how props and state control UI
- how to build forms and handle events
- how to render lists and conditions
- how SPA routing works with React Router
- how to connect to REST APIs
- how to manage loading and error states
- how JWT authentication works at a high level

Useful Links

- ▶ https://www.tutorialspoint.com/reactjs/reactjs_introduction.htm
- ▶ <https://www.w3sch>
- ▶ <https://www.geeksforgeeks.org/reactjs/react/ools.com/REACT/DEFAULT.ASP>